

Joseph Cassello Jr

CS 499 Enhancement Three: Databases

Professor Phillips

February 8, 2026

Artifact Description:

The artifact I chose for this enhancement is the Animal Shelter Project, which I originally developed during the CS 340 Client/Server Development course. At its core, the project is a full stack application designed to help a search and rescue organization named Grazioso Salvare identify a specific dog breed for training. It was built using a Python module that interfaces with a MongoDB database and uses a dashboard to display data about the animals. In its initial state, the software allowed for basic create, read, update, and delete operations, but it functioned primarily as a local script intended for a single user environment.

Justification for Inclusion:

I chose to include this specific project because it showcases my ability to manage complex data while bridging the gap between back end logic and front end visualization. It is a prime example of my work in NoSQL database management and Python programming. I selected this item for enhancement because converting a local script into a networked service is a significant leap in complexity that demonstrates professional growth.

To improve the artifact, I transformed the database module into a robust RESTful Web API using the Flask framework. This change allows the system to handle requests over a network, which is an essential skill in modern software development. I also significantly upgraded the dashboard logic to ensure that the interact maps and breed distribution charts render

correctly without being cut off or failing to update. These components show that I can take a functional but limited prototype and evolve it into a scalable, high quality application that follows industry standards.

Alignment with Course Outcomes:

The enhancements made to this project directly satisfy the two course outcomes I targeted at the start of this milestone. First, I met Course Outcome 4 by utilizing well founded architectural techniques to build a REST API. This move from a local script to a service oriented architecture shows I can deliver solutions that provide real values in a professional, multi user context. By structuring the code into separate modules for the database, the API service, and the user interface, I have created a system that is far easier to maintain and scale.

Second, I met Course Outcome 5 by implementing a validation layer within the API. This demonstrates a security mindset because the code now acts as a gatekeeper that inspects all incoming data. By checking for missing fields or logical errors like negative age values, I have mitigated design flaws that could lead to data corruption or system instability. My outcome coverage plans remain the same, as the completed work mirrors my original goals for security and professional engineering.

Reflection on the Process:

Reflecting on the process of enhancing this artifact, I realized that moving data between different systems requires a much higher level of precision than working with local files. One of the most important things I learned was how to handle data serialization effectively. For example, I had to ensure that the unique IDs used by MongoDB were properly cleaned before being sent to the dashboard to prevent the entire interface from crashing.

The biggest challenge I faced during this process was managing the communication between the data table and visual components. In the beginning, the map and the pie chart were not showing up because the unique IDs in the code did not match the signals being sent by the user's clicks. Debugging these "silent features" taught me to be much more disciplined in how I name and track components across a large project. Overcoming these hurdles made me a more capable developer because I had to move beyond just making the code work and focus on making it resilient, secure, and user friendly.